NVIDIA H100 80GB HBM3 (SM_90A) · CUDA 12.8 · PYTORCH 2.8 · TRITON 3.4 · NCCL 2.27.3

Long-Context LLM Decode Engine: Architecture, Bottlenecks & Negative Results

End-to-end systems evaluation across custom CUDA (`v4`, Tensor-Core `cp.async`), Triton split-KV & Quest 64K sparse kernels, per-page FP8 KV quantization, `Qwen/Qwen3-8B` CUDA-Graph serving vs FlashInfer & vLLM, `nsys`/SASS attribution, and multi-GPU `TP=2/4` NVLink scaling.

2903 GB/s	2968.5 tok/s	7.55x	2.02x
Peak Kernel BW (86.6% H100 peak, B=4 64K FP16)	Qwen3-8B E2E FP8 at B=64 (+16% & 0.5x KV vs FP16)	TP=4 CUDA Graph over NCCL (0.156 ms/layer B=1)	Lossless 64K Quest Sparse (25% budget, +0.19% PPL)

1. Kernel Optimization & Falsified Hypotheses (WP-0, WP-3)

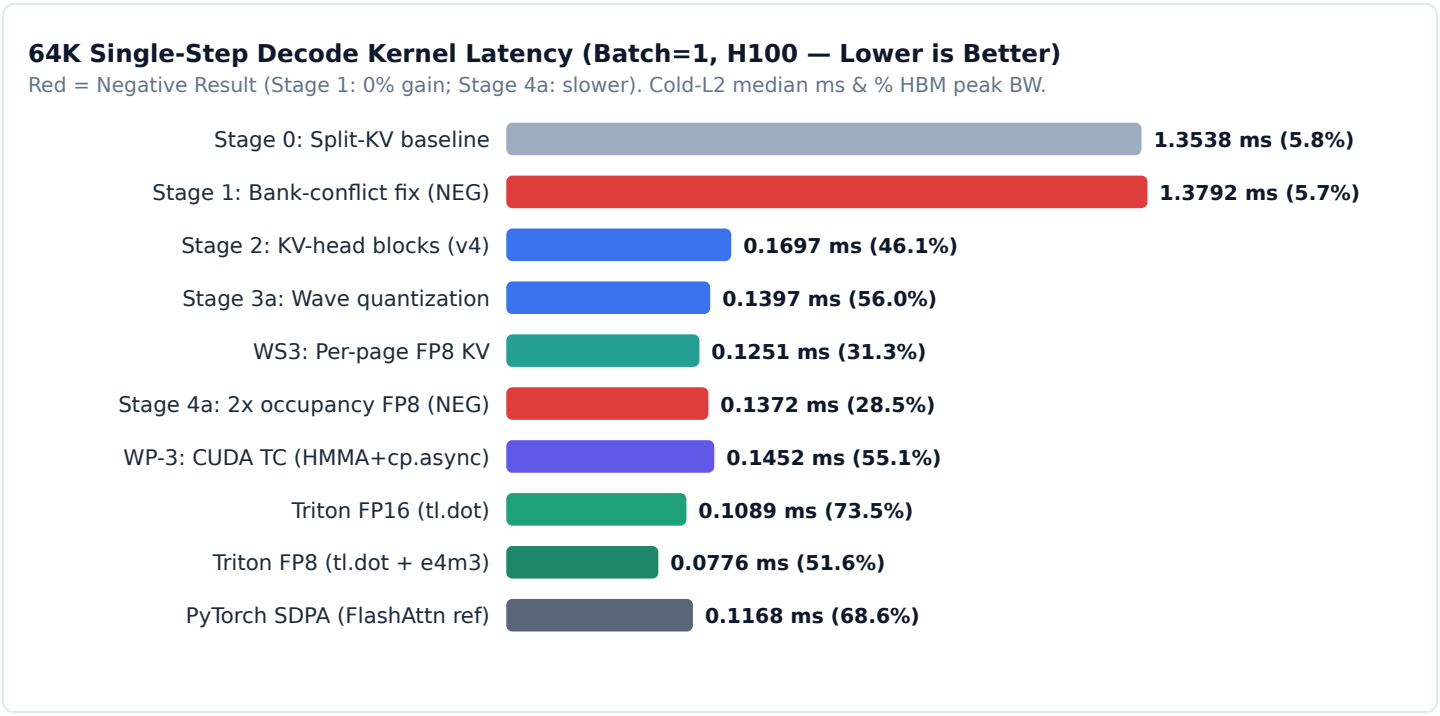
Starting from a naive split-KV CUDA baseline (`1.3538 ms` at 64K, 5.8% of HBM peak), staged experiments isolated the true hardware bottlenecks on H100 SXM (`Hq=32, Hkv=8, D=128`):

Negative Result #1 — Shared-Memory Bank-Conflict Removal Bought 0% (Stage 1)

Removing a verified 32-way shared-memory bank conflict on `tile_K[tid][d]` moved 64K latency from `1.3538 ms` to `1.3792 ms` (5.7% HBM peak). At ~194 GB/s, the kernel was latency-starved and reading K/V **4x redundantly** across query-head blocks. Grouping all 4 query heads per KV-head block and shrinking shared tiles (Stage 2, `37.5%` occupancy) cut latency **8.0x** to `0.1697 ms` , and wave quantization (Stage 3a) reached `0.1397 ms` .

Negative Result #2 — Doubling Occupancy Slowed Both FP16 & FP8 (Stage 4a)

Once per-page FP8 (`ws3`) halved DRAM traffic (`0.1251 ms`), HBM utilization dropped from 56% to 31%. Halving tile size to double occupancy (`37.5% -> 75%`) regressed latency to `0.1372 ms` (FP8) and `0.1474 ms` (FP16), proving the scalar FMA CUDA kernel was **instruction-issue bound** (~1,700 ops/lane), not occupancy- or bandwidth-bound. Moving QK^T and PV onto Tensor Cores — via CUDA `HMMA + cp.async` (`gqa_decode_tc.cu` , zero scalar FMA in FP16 SASS) and Triton `tl.dot` — resolved the issue floor, hitting `0.0776 ms` (FP8 B=1 64K) and `2903 GB/s` (FP16 B=4 64K).



64K Batch=1 Kernel Latency Progression (ms, Cold L2)

Stage / Kernel	Ours (ms)	SDPA (ms)	Ratio	% Peak	Max Err	Commit
stage0-baseline	1.3538	0.1015	13.34x	5.8%	0.00002	6779c18
stage1-bank-conflict	1.3792	0.1013	13.62x	5.7%	0.00002	caf92f4
stage2-v4-kvhead-blocks	0.1697	0.1017	1.67x	46.1%	0.00002	c1da13a
stage3a-wave-quantization	0.1397	0.1017	1.37x	56.0%	0.00002	a5df7ff
ws3-fp8-kv	0.1251	0.1022	1.22x	31.3%	0.00092	d2dfaee
stage4a-tile32-fp16	0.1474	0.1016	1.45x	53.1%	0.00002	1667b0b
stage4a-tile32-fp8	0.1372	0.1026	1.34x	28.5%	0.00092	1667b0b
WP-3 cuda_v4	0.1540	-	-	52.0%	0.00002	ab46cf5
WP-3 cuda_tc	0.1452	-	-	55.1%	0.00002	ab46cf5
WP-3 cuda_tc_fp8	0.1181	-	-	33.9%	0.00100	ab46cf5
WP-3 triton_fp16	0.1089	-	-	73.5%	0.00002	ab46cf5
WP-3 triton_fp8	0.0776	-	-	51.6%	0.00101	ab46cf5
WP-3 sdpa	0.1168	-	-	68.6%	0.00000	ab46cf5

Compiled CUDA SASS Instruction Summary (sm_90a)

Kernel	HMMA	LDGSTS	LDG.E.128	LDG	LDS	LDSM	FFMA/HFMA2	Total
gqa_decode_v4_fp8_kernel	0	0	2	9	232	0	262	1984
gqa_decode_v4_kernel	0	0	2	7	232	0	260	1896
gqa_combine_kernel	0	0	0	44	0	0	10	304
gqa_decode_splitkv_kernel	0	0	0	15	103	0	94	1144
gqa_decode_tc_kernel<128,64,true>	32	34	5	7	171	8	66	2784
gqa_decode_tc_kernel<128,64,false>	32	34	5	5	75	24	2	2520
gqa_decode_tc_kernel<64,64,true>	16	34	5	7	107	4	34	2432
gqa_decode_tc_kernel<64,64,false>	16	34	5	5	59	12	2	2304

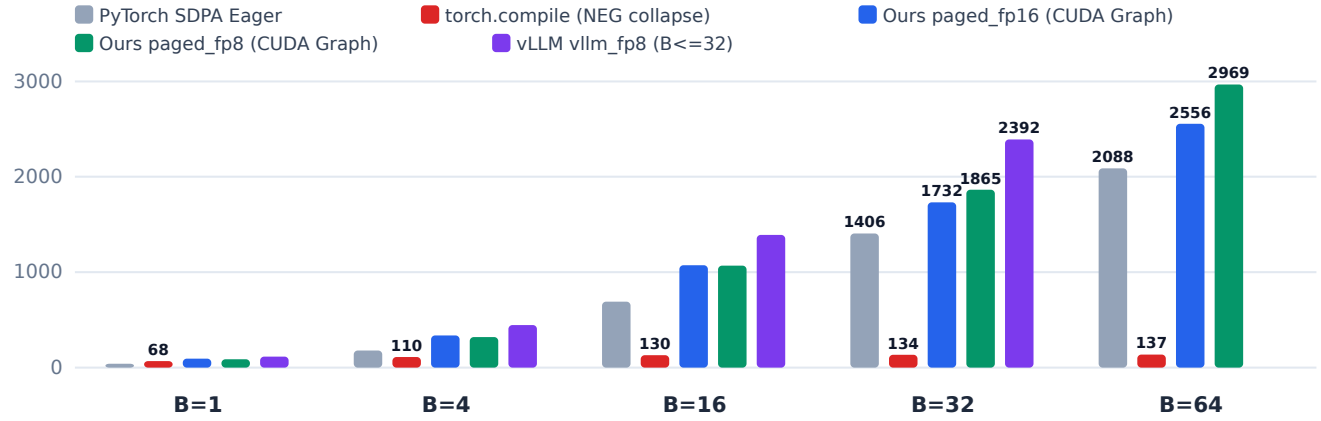
Kernel	HMMA	LDGSTS	LDG.E.128	LDG	LDS	LDSM	FFMA/HFMA2	Total
gqa_combine_tc_kernel	0	0	0	44	0	0	10	336

2. End-to-End Qwen/Qwen3-8B Serving vs. PyTorch, FlashInfer & vLLM (WP-2)

Full-step CUDA Graphs over static paged KV pools (`block_size=64`, `torch.library.custom_op`) reduce single-user `Qwen3-8B` 4K ITL from `25.24 ms` (eager) to **10.68 ms** (**2.36x**) and saturate batch 64 at **2968.5 tok/s** with FP8 KV.

End-to-End Qwen/Qwen3-8B Decode Throughput (ctx=4096, tok/s — Higher is Better)

CUDA Graphs scale linearly to 2,968.5 tok/s (FP8 at B=64); torch.compile collapses to 136.8 tok/s.



Negative Result #3 — torch.compile Collapse on Dynamic KV Shapes (sdpa_compile)

While `sdpa_compile` helps at `B=1, 4K` (`14.67 ms` vs `25.24 ms` eager), dynamic KV shapes and recompilation/fallback overhead cause severe degradation at scale: `122.91 ms` at `B=16, 4K`, **467.98 ms (136.8 tok/s)** at `B=64, 4K`, and **565.83 ms** at `B=4, 64K`. Explicit `torch.library.custom_op` custom paged kernels + static CUDA Graphs scale monotonically without graph breaks.

Bottleneck Shift — FP8 Crossover Point & Synthetic 4K Prompt Tie Divergence

At low KV pressure (`B=1, 4K`; `0.62 GB` FP16 KV), per-token FP8 quantize/rescale overhead makes `paged_fp8` slightly slower than `paged_fp16` (`11.53 ms` vs `10.68 ms`). Once KV traffic dominates HBM at `B=64, 4K`, **FP8 flips to +16% faster** (`2968.5` vs `2556.3 tok/s`) while cutting KV cache from **39.86 GB to 19.94 GB**. Separately, `paged_fp8` matches `73/73` greedy tokens at `16K` and `64K` and passes real-text `8K` PPL (`-0.07%`), but diverges at step 2 (`agree=2`) on the low-entropy synthetic repeated `4K` prompt.

Kernel Head-to-Head vs. FlashInfer (BatchDecodeWithPagedKVCache, CUDA Graph)

Method	Batch	Ctx	Ours (ms)	FlashInfer (ms)	Ratio
kernel_fp16	1	4096	0.0094	0.0127	0.741x
kernel_fp8	1	4096	0.0088	0.0167	0.529x
kernel_fp16	16	4096	0.0974	0.0993	0.981x
kernel_fp8	16	4096	0.0695	0.1384	0.502x
kernel_fp16	32	4096	0.1844	0.1886	0.978x
kernel_fp8	32	4096	0.1184	0.2505	0.473x
kernel_fp16	1	16384	0.0308	0.0334	0.921x
kernel_fp8	1	16384	0.0187	0.0450	0.416x
kernel_fp16	16	16384	0.3606	0.3702	0.974x

Method	Batch	Ctx	Ours (ms)	FlashInfer (ms)	Ratio
kernel_fp8	16	16384	0.2493	0.4737	0.526x
kernel_fp16	32	16384	0.7082	0.7226	0.980x
kernel_fp8	32	16384	0.4407	0.9438	0.467x
kernel_fp16	1	65536	0.0975	0.1003	0.972x
kernel_fp8	1	65536	0.0830	0.1276	0.651x
kernel_fp16	16	65536	1.4085	1.4480	0.973x
kernel_fp8	16	65536	0.9587	1.8394	0.521x
kernel_fp16	32	65536	2.8201	2.8694	0.983x
kernel_fp8	32	65536	1.7193	3.7251	0.462x

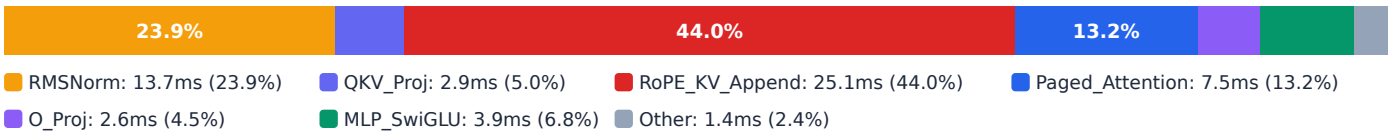
End-to-End Qwen3-8B Throughput vs. vLLM (tok/s, vLLM skipped at B=64 / 64K OOM budget)

Batch	Ctx	Ours FP16 (tok/s)	Ours FP8 (tok/s)	vLLM FP16 (tok/s)	vLLM FP8 (tok/s)
1	4096	93.6	86.7	123.3	115.0
16	4096	1074.5	1068.7	1367.9	1390.4
32	4096	1733.0	1865.7	2120.1	2391.6
1	16384	87.4	83.7	108.8	106.7
16	16384	657.4	742.9	749.5	862.0
1	65536	72.0	70.1	85.8	96.7

3. Root-Cause Attribution: Eager Dispatch Wall & Residual Gap vs. vLLM (WP-5)

Real `nsys stats` profiling (NVTX ranges + `cuda_gpu_kern_sum`) explains both why CUDA Graphs deliver 2.36x E2E speedup and where the remaining ~22% gap vs. production vLLM (`10.68 ms` vs `8.11 ms` at `B=1, 4K`) originates.

Eager Decode Step NVTX Breakdown (Qwen3-8B, paged_fp8, B=16, 4K — Total: 57.09 ms/step)



The Dispatch Illusion: 68% Host Overhead Masking 1.1–3.7 us GPU Kernels

NVTX wall-time attribution on eager `Qwen/Qwen3-8B` measures `57.1–59.6 ms/step`, dominated by `RoPE_KV_Append` (**43.3–44.0%**) and `RMSNorm` (**23.9–24.8%** across 109 calls), while `Paged_Attention` is only **12.7–13.2%**. Yet `cuda_gpu_kern_sum` reveals those PyTorch elementwise/reduce kernels execute on-GPU in just **1.1–3.7 us per call**. Nearly 80% of eager latency is Python/CUDA launch overhead — eliminated by CUDA Graphs — while the remaining ~22% gap vs. vLLM comes from running 109+ unfused kernels inside the graph instead of fused `RMSNorm + RoPE + KV Store`.

Eager Decode Step NVTX Wall-Time Attribution (ms / share of step)

Stage	B=1, ctx=4K	B=16, ctx=4K	B=64, ctx=4K	B=1, ctx=64K
Embedding	0.039 ms (0.1%)	0.033 ms (0.1%)	0.045 ms (0.1%)	0.036 ms (0.1%)
RMSNorm	14.546 ms (24.5%)	13.651 ms (23.9%)	14.770 ms (24.8%)	14.618 ms (24.7%)
QKV_Proj	3.086 ms (5.2%)	2.868 ms (5.0%)	2.911 ms (4.9%)	3.064 ms (5.2%)
RoPE_KV_Append	25.705 ms (43.3%)	25.130 ms (44.0%)	26.159 ms (43.9%)	25.662 ms (43.3%)

Stage	B=1, ctx=4K	B=16, ctx=4K	B=64, ctx=4K	B=1, ctx=64K
Paged_Attention	7.684 ms (13.0%)	7.510 ms (13.2%)	7.600 ms (12.7%)	7.622 ms (12.9%)
O_Proj	2.702 ms (4.6%)	2.597 ms (4.5%)	2.438 ms (4.1%)	2.694 ms (4.5%)
MLP_SwiGLU	4.064 ms (6.9%)	3.891 ms (6.8%)	4.182 ms (7.0%)	4.068 ms (6.9%)
LM_Head	0.036 ms (0.1%)	0.034 ms (0.1%)	0.038 ms (0.1%)	0.037 ms (0.1%)
Other	1.462 ms (2.5%)	1.378 ms (2.4%)	1.483 ms (2.5%)	1.439 ms (2.4%)
Decode_Step (Total)	59.325 ms (100.0%)	57.094 ms (100.0%)	59.625 ms (100.0%)	59.240 ms (100.0%)

4. Long-Context 64K Quest Sparsity & The Sparsity-Quality Cliff (WP-4)

Two-stage Quest page selection (page min/max upper-bound scoring + sink page 0 + local window) genuinely reduces 64K attention HBM traffic proportional to `top_k_pages / total_pages` (`14.1–28.2%` of dense bytes):

Negative Result #4 — Quest@12.5% Passes 3/3 Needle Recall Yet Fails PPL (+7.66%)

At 64K (`block_size=64`), aggressive `12.5%` page selection cuts attention bytes to **14.1%** and delivers **3.14x** FP16 speedup while achieving **3/3 Needle-in-a-Haystack** retrieval at 10/50/90% depths. However, teacher-forced perplexity on `Qwen/Qwen3-8B` regresses by **+7.66% (FAIL)**. Increasing budget to **25.0%** passes the quality gate (**+0.19%** PPL delta, **100%** confident-step parity, 3/3 needles) while retaining **2.02x** (FP16) and **1.78x** (FP8) kernel speedups.

64K Quest Attention Kernel Speedup & HBM Traffic (B=4, CUDA Graph)

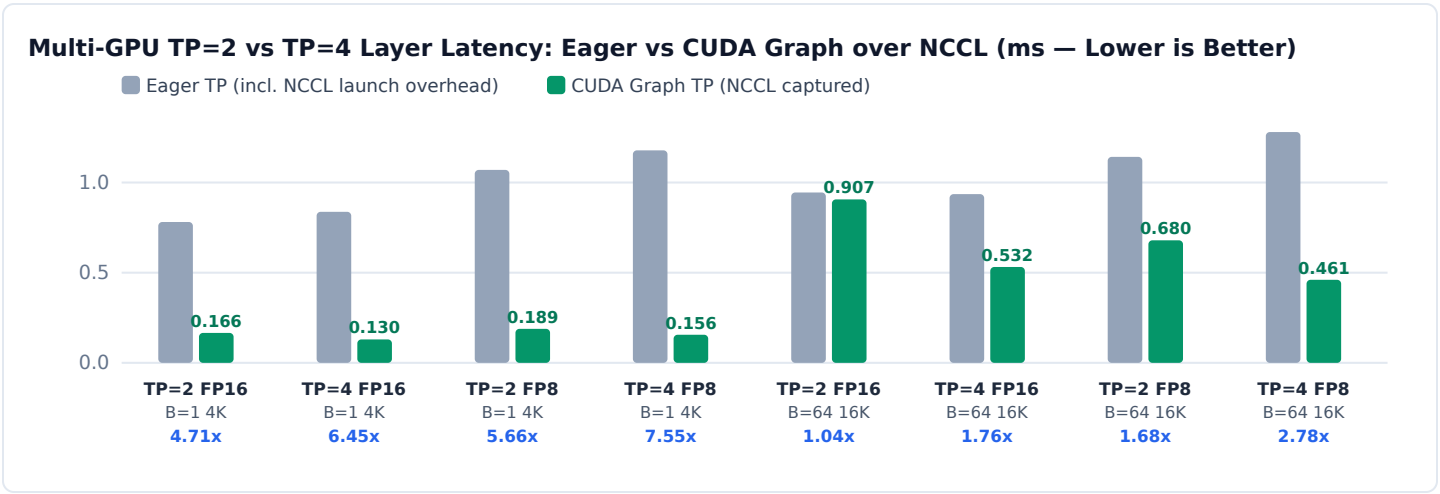
KV	Block	Budget	Dense (us)	Quest (us)	Speedup	HBM Bytes
fp16	16	12.5%	436.5	211.9	2.06x	18.9%
fp16	16	25.0%	436.5	299.3	1.46x	31.4%
fp16	64	12.5%	376.8	119.9	3.14x	14.1%
fp16	64	25.0%	376.8	186.8	2.02x	26.6%
fp8	64	12.5%	322.1	115.9	2.78x	15.7%
fp8	64	25.0%	322.1	180.9	1.78x	28.2%

Qwen/Qwen3-8B Quality Gate (64K Context, PPL ≤ 1.0% Threshold)

Path	Budget	PPL Delta	Confident Parity	Needle Recall	Verdict
paged	100%	-0.02%	100.0%	3/3	PASS
quest@0.125	12.5%	+7.66%	100.0%	3/3	FAIL
quest@0.25	25.0%	+0.19%	100.0%	3/3	PASS

5. Multi-GPU Tensor Parallelism (TP=2 & TP=4) Over NVLink (WP-5)

Real `torch.distributed` NCCL `2.27.3` benchmarks on `H100!:2` and `H100!:4` prove that NCCL `AllReduce` captures cleanly inside CUDA Graphs (`nccl_graph_supported=true`), collapsing single-layer TP decode by **4.71x–5.66x (TP=2)** and **6.36x–7.55x (TP=4)**:



Why Eager TP Communication Was 24–39% Despite 8.0 us NCCL GPU Kernels

`nsys` on `TP=2` measures the actual NVLink kernel `ncclDevKernel_AllReduce_Sum_f16_RING_LL` at **8.0 us median** (`0.3%` of trace), while eager NVTX ranges `TP_AllReduce_Attn` (**93.9 us** avg) and `TP_AllReduce_MLP` (**71.8 us** avg) inflate communication overhead to `20–39%` of eager step time. Capturing the TP block in a CUDA Graph reduces `B=1, 4K` latency to **0.130 ms (TP=4 FP16)** and **0.156 ms (TP=4 FP8)**, and scales `B=64, 16K` FP8 down to **0.461 ms** (1.48x faster than `TP=2 FP8` and 1.97x faster than `TP=2 FP16`).

1-Layer Qwen3-8B TP=2 & TP=4 Latency Over NVLink (Batch=1, 16, 64)

TP	KV Mode	Batch	Ctx	Compute (ms)	Eager (ms)	Graph (ms)	Speedup	Comm %
2	paged_fp16	1	4096	0.581	0.781	0.166	4.71x	25.6%
2	paged_fp16	16	4096	0.620	0.826	0.240	3.43x	24.9%
2	paged_fp16	64	4096	0.623	0.818	0.380	2.15x	23.8%
2	paged_fp16	1	16384	0.593	0.805	0.173	4.65x	26.3%
2	paged_fp16	16	16384	0.640	0.843	0.370	2.28x	24.1%
2	paged_fp16	64	16384	0.921	0.945	0.907	1.04x	2.6%
2	paged_fp8	1	4096	0.854	1.070	0.189	5.66x	20.3%
2	paged_fp8	16	4096	0.897	1.140	0.261	4.36x	21.3%
2	paged_fp8	64	4096	0.888	1.100	0.353	3.12x	19.2%
2	paged_fp8	1	16384	0.839	1.085	0.198	5.49x	22.7%
2	paged_fp8	16	16384	0.896	1.134	0.369	3.07x	21.0%
2	paged_fp8	64	16384	0.904	1.143	0.680	1.68x	20.9%
4	paged_fp16	1	4096	0.581	0.838	0.130	6.45x	30.7%
4	paged_fp16	16	4096	0.700	0.918	0.191	4.81x	23.8%
4	paged_fp16	64	4096	0.594	0.850	0.270	3.15x	30.1%
4	paged_fp16	1	16384	0.623	0.885	0.139	6.36x	29.7%
4	paged_fp16	16	16384	0.609	0.971	0.258	3.77x	37.3%
4	paged_fp16	64	16384	0.617	0.936	0.532	1.76x	34.1%
4	paged_fp8	1	4096	0.958	1.178	0.156	7.55x	18.7%
4	paged_fp8	16	4096	0.874	1.206	0.214	5.63x	27.5%
4	paged_fp8	64	4096	0.863	1.288	0.279	4.62x	33.0%
4	paged_fp8	1	16384	0.807	1.159	0.161	7.22x	30.4%

TP	KV Mode	Batch	Ctx	Compute (ms)	Eager (ms)	Graph (ms)	Speedup	Comm %
4	paged_fp8	16	16384	0.865	1.227	0.276	4.45x	29.5%
4	paged_fp8	64	16384	1.003	1.280	0.461	2.78x	21.6%

6. Honest Scope, Limitations & Next Engineering Levers

- **Nsight Compute (`ncu`) skipped on Modal:** Modal container permissions block hardware PMU counters (`ERR_NVGPUCTRPERM`). Instead, warp/pipe stalls and instruction mix are anchored via `cuobjdump -sass` (`results/profiles/h100_sass.md`), `nsys` GPU kernel summaries, and cold-L2 CUDA event timings.
- **Single-node VRAM & E2E TP Scope:** `Qwen3-8B` single-GPU serving hits HBM capacity limits at `B>=16, 64K` (`need 311.6 GB` at `B=64, 64K` vs `66.4 GB` free). Meanwhile, `bench_tp_decode.py` benchmarks a 1-layer TP shard with real NCCL collectives rather than a 36-layer sharded HF serving loop.
- **Next Highest-ROI Steps:** (1) Fuse `RMSNorm + RoPE + Paged FP8/MinMax Store` to remove ~100 small kernel launches inside the CUDA Graph and close the remaining ~22% gap vs. vLLM; (2) wire Quest `25%` sparse attention + FP8 KV across sharded 36-layer `TP=4` E2E serving to unlock `B=64, 64K` fits.